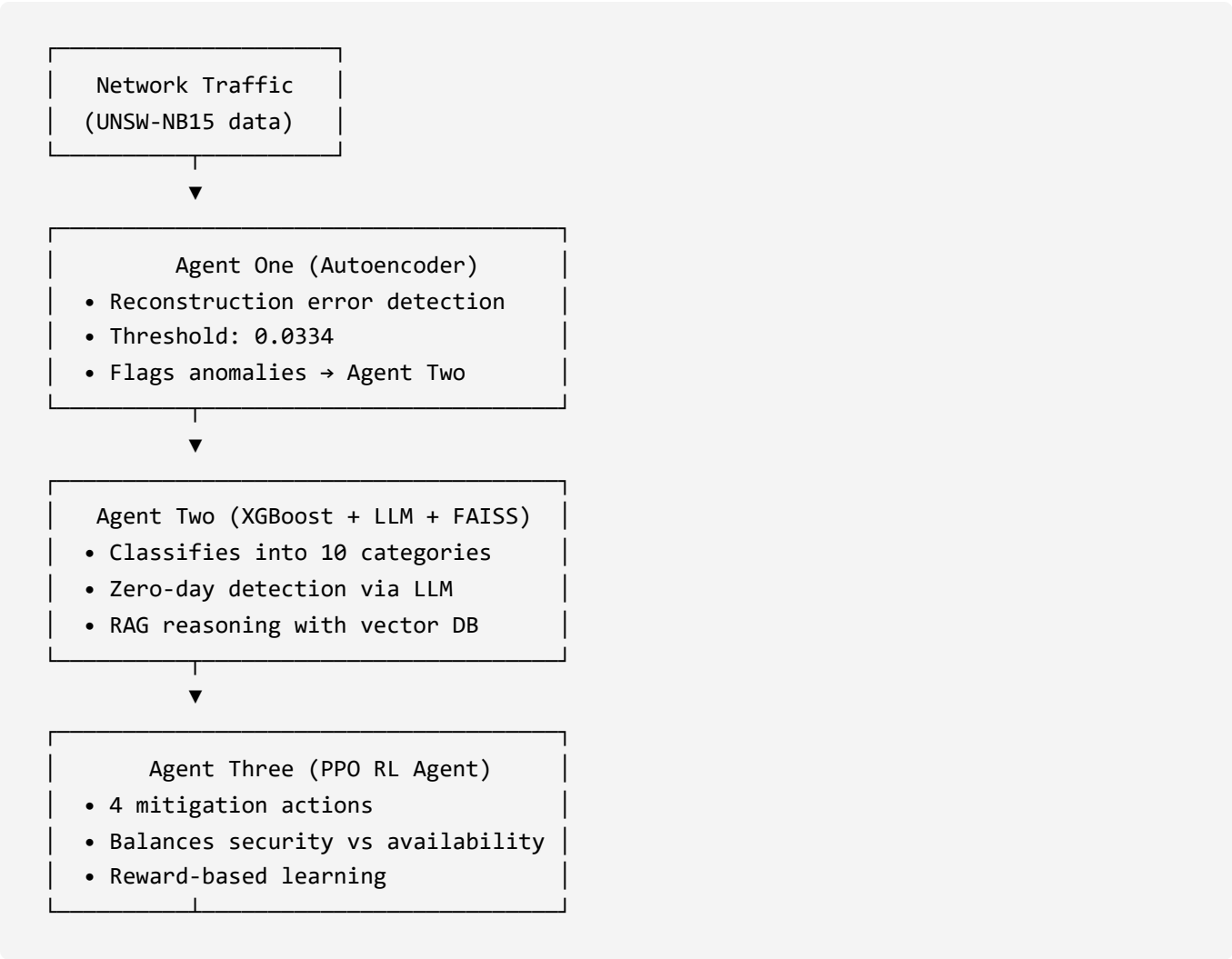


System Overview

A three-agent system for network intrusion detection using the **UNSW-NB15** dataset:

Agent	Role	Technology	Status
Agent One	Anomaly Detection	PyTorch Autoencoder	Trained (100% recall)
Agent Two	Classification & Reasoning	XGBoost + LangChain/LLM	Trained (83.21% accuracy)
Agent Three	Mitigation Decision	PPO (Stable Baselines 3)	Trained (93.1% accuracy)

Architecture Flow



Installation

```
# Clone the repo (if not already done)
cd F:\FYP\Cyber-Threat-Intelligence

# Create and activate virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Install all dependencies
pip install -r requirements.txt
```

1. Data Pipeline

Components

- **DataLoader**: Loads UNSW-NB15 CSV files, handles dtypes
- **Preprocessor**: Encodes categorical features, scales numerical features
- **BatchGenerator**: Yields batches for training

2. Agent One: Anomaly Detection (Autoencoder)

- Trains an autoencoder on **normal traffic only**
- Normal traffic → low reconstruction error
- Attacks → high reconstruction error (anomaly)
- Threshold-based classification: **error > threshold → anomaly**

Training Command

```
python -m agents.train_autoencoder --data-path data/UNSW_NB15_training-set.csv --epochs 50
```

Model Architecture

- Encoder: 196 → 128 → 64 → 32 (latent)
- Decoder: 32 → 64 → 128 → 196
- Activation: LeakyReLU, Dropout
- Loss: MSE (reconstruction error)

Why Keep Agent One?

Factor Analysis

- Computational efficiency Autoencoder is ~10x faster than XGBoost classification. At 100K flows/sec, filtering 50% with Agent One saves massive compute.
- Zero-day detection Autoencoder detects any anomaly, even novel attacks XGBoost wasn't trained on.
- Current state Agent One only missed 22 attacks (1.35%). That's excellent recall.

3. Agent Two: Classification & LLM Reasoning

1. **XGBoost Classifier:** Classifies threats into 10 categories:
 - Normal, Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms
2. **Zero-Day Detection:** Low-confidence predictions trigger LLM reasoning
3. **RAG Pipeline:** FAISS vector DB retrieves similar attacks for context
4. **LLM Reasoning:** Generates analysis for unknown threats

Training Command

```
python -m agents.train_xgboost --data-path data/UNSW_NB15_training-set.csv --init-vector-db
```

Inference Code

PYTHON

```
from agents import AgentTwo
from agents.interfaces import FAISSVectorDB, MockLLM # or OllamaLLM for real LLM

# Load components
agent = AgentTwo.from_pretrained("models/agent_two")

# Analyze a threat (using preprocessed features)
import numpy as np
features = np.random.rand(196).astype(np.float32)

result = agent.analyze_threat(features)
print(result.summary)

# Access individual fields
print(f"Category: {result.classification.predicted_category}")
print(f"Confidence: {result.classification.confidence:.2%}")
print(f"Is Zero-Day: {result.is_zero_day}")
print(f"Severity: {result.severity}")
print(f"Recommended Actions: {result.recommended_actions}")
```

With Real LLM (Ollama)

PYTHON

```
from agents.interfaces import OllamaLLM

# Create with Ollama
agent = AgentTwo.from_pretrained(
    "models/agent_two",
    llm=OllamaLLM(model="llama3", base_url="http://localhost:11434")
)
```

4. Agent Three: RL Mitigation Agent

- **Environment:** Custom Gymnasium environment (`NetworkDefenseEnv`)
- **State:** 14-dim vector (category one-hot + confidence + severity + zero_day + high_risk)
- **Actions:** 4 discrete mitigation actions
 - `DO_NOTHING (0)`: Continue monitoring
 - `ALERT_ADMIN (1)`: Send alert to security team
 - `BLOCK_IP (2)`: Block source IP address
 - `ISOLATE_SUBNET (3)`: Isolate affected subnet
- **Reward Function:**
 - True Positive: +10 to +50 (scaled by severity)
 - True Negative: +5
 - False Positive: -20 to -80 (penalize disrupting normal traffic)
 - False Negative: -30 to -90 (penalize missing threats)

Training Command

```
python -m agents.train_rl --timesteps 100000 --save-path models/agent_three
```

Inference Code

PYTHON

```
from agents import AgentThree, ThreatState, MitigationAction

# Load trained agent
agent = AgentThree.from_pretrained("models/agent_three")

# Option 1: From ThreatState directly
threat = ThreatState(
    category_index=4,    # DoS
    confidence=0.88,
    severity_level=2,    # high
    is_zero_day=False,
    is_actual_threat=True
)
decision = agent.take_action(threat)
print(f"Action: {decision.action_name}")
print(f"Confidence: {decision.confidence:.2%}")
print(f"\nReasoning:\n{decision.reasoning}")

# Option 2: From dict (e.g., Agent Two output)
threat_dict = {
    "category": "Exploits",
    "confidence": 0.92,
    "severity": "critical",
    "is_zero_day": True,
}
decision = agent.take_action(threat_dict)
print(f"Action: {decision.action_name}")

# Option 3: From Agent Two's ThreatAnalysisResult
# decision = agent.take_action(agent_two_result)
```

5. Full Pipeline Integration

Run Integration Test

```
python tests/test_integration.py --data-path data/UNSW_NB15_training-set.csv
```

Complete Pipeline Code

PYTHON

```
from data_pipeline import DataLoader, Preprocessor
from agents import AgentOne, AgentTwo, AgentThree, ThreatState
import numpy as np

# 1. Load and preprocess data
loader = DataLoader("data/UNSW_NB15_training-set.csv")
X, y = loader.load()
preprocessor = Preprocessor()
preprocessor.load("models/preprocessor.pkl")
X_processed, y_binary, y_category = preprocessor.transform(X, y)

# 2. Agent One: Detect anomalies
agent_one = AgentOne.from_checkpoint("models/agent_one/best_model.pth",
threshold=0.0334)
results = agent_one.detect_anomalies(X_processed[:100])
anomaly_indices = [i for i, r in enumerate(results) if r.is_anomaly]
print(f"Anomalies detected: {len(anomaly_indices)}")

# 3. Agent Two: Classify anomalies
agent_two = AgentTwo.from_pretrained("models/agent_two")
for idx in anomaly_indices[:5]: # First 5 anomalies
    analysis = agent_two.analyze_threat(X_processed[idx])
    print(f"\nSample {idx}:")
    print(f"  Category: {analysis.classification.predicted_category}")
    print(f"  Severity: {analysis.severity}")

# 4. Agent Three: Decide mitigation
agent_three = AgentThree.from_pretrained("models/agent_three")
decision = agent_three.take_action(analysis)
print(f"  Mitigation: {decision.action_name} ({decision.confidence:.1%})")
```

6. Project Structure

```
Cyber-Threat-Intelligence/
├── agents/
│   ├── __init__.py          # Exports all agents
│   ├── agent_one.py         # Autoencoder anomaly detection
│   ├── agent_two.py         # XGBoost + LLM classification
│   ├── agent_three.py       # PPO RL mitigation agent
│   ├── train_autoencoder.py  # Training script for Agent One
│   ├── train_xgboost.py     # Training script for Agent Two
│   ├── train_rl.py          # Training script for Agent Three
│   └── models/
│       ├── autoencoder.py    # AnomalyAutoencoder class
│       └── xgboost_classifier.py # ThreatClassifier class
│   └── environments/
│       └── network_defense_env.py # Gymnasium RL environment
│   └── interfaces/
│       └── base.py           # LLM, VectorDB interfaces
├── data_pipeline/
│   ├── __init__.py
│   ├── data_loader.py        # DataLoader class
│   ├── preprocessor.py       # Preprocessor class
│   ├── batch_generator.py    # BatchGenerator class
│   └── config.py             # Dataset configuration
├── models/                   # Saved trained models
│   ├── agent_one/
│   ├── agent_two/
│   ├── agent_three/
│   └── preprocessor.pkl
├── data/
│   └── UNSW_NB15_training-set.csv
├── tests/
│   └── test_integration.py    # Integration test
└── requirements.txt
```

7. Performance Summary

Agent	Metric	Value
Agent One	Recall	100%
Agent One	Threshold	0.0334
Agent Two	Accuracy	83.21%
Agent Three	Accuracy	93.1%

Agent	Metric	Value
Agent Three	Precision	92.8%
Agent Three	Recall	97.8%
Agent Three	F1 Score	95.2%

Questions for Clarification

1. **LLM Integration:** Are you using **Ollama locally** for the LLM, or do you want to integrate with **OpenAI API** or another provider?
2. **Dataset:** Do you have the full **UNSW-NB15** dataset in data? The training scripts expect `UNSW_NB15_training-set.csv`.
3. **Real-Time Operation:** Should the system work in **real-time** (streaming) or **batch mode** for now?
4. **Agent Three Actions:** Should the mitigation actions (Block IP, Isolate Subnet) actually execute commands, or just output recommendations?
5. **Web Interface:** Do you need a **dashboard/UI** to visualize detections and decisions?
6. **Deployment:** Where will this be deployed? Local machine, cloud, or integrated with existing security tools (SIEM)?
7. **Additional Categories:** UNSW-NB15 has 10 categories. Do you need to add **custom threat types** beyond these?